ĐẠI HỌC QUỐC GIA HÀ NỘI

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ



BÁO CÁO THỰC TẬP

NGÀNH: CÔNG NGHỆ THÔNG TIN

ĐỀ TÀI: PHÁT TRIỂN VÀ TỐI ƯU TÍNH NĂNG CHO HỆ THỐNG QUẢN LÝ KHÁCH SẠN KIOTVIET HOTEL

Cán bộ hướng dẫn: Trần Mạnh Cường

Giảng viên đánh giá: [chờ bổ sung]

Sinh viên: Trần Quang Thành

Mã sinh viên: 23020155

Lớp: K68I-IT2

Hà Nội, tháng 9 năm 2026

MỤC LỤC

I. Giới thiệu chung 1
- a. Giới thiệu công ty 1
- b. Giới thiệu công việc 1
- c. Giới thiệu qua bài toán 1

II. Yêu cầu bài toán 2
- a. Miêu tả chi tiết bài toán 2
- b. Phần bài toán em trực tiếp giải quyết 3
- c. Công việc của nhóm và công việc của em 4

III. Tóm tắt lý thuyết, giải pháp, thuật toán 5
- a. Các lý thuyết và giải pháp liên quan 5
- b. Cách giải quyết của em 6
- c. Liên hệ và so sánh với các cách đã có 11

IV. Mô tả phần mềm cài đặt 12
- a. Phạm vi trình bày 12
- b. Môi trường và công cụ cần có 12
- c. Cách tổ chức mã nguồn và cách dựng phần mềm 12
- d. Cách đưa thay đổi cơ sở dữ liệu 13
- e. Chuỗi môi trường và quy trình phát hành 13
- f. Kiểm thử tự động trong quy trình 14

V. Kết quả đạt được, hướng phát triển 14
- a. Kỹ năng và kiến thức thu thập được 14
- b. Hướng phát triển tiếp theo 17

Tài liệu tham khảo 18

LỜI CẢM ƠN

Sáu tháng làm việc tại KiotViet là quãng thời gian em học được nhiều hơn tất cả những gì em hình dung trước khi bước vào kỳ thực tập. Trước ngày đầu tiên, em vẫn nghĩ đi thực tập là được giao một bài tập nhỏ, làm xong thì nộp. Thực tế khác hẳn: em được đặt vào một hệ thống đang chạy thật, có rất nhiều cơ sở lưu trú dùng mỗi ngày, và mỗi dòng mã em viết đều có người ở đầu bên kia chờ dùng.

Em xin gửi lời cảm ơn chân thành tới anh Trần Mạnh Cường, cán bộ hướng dẫn trực tiếp của em tại công ty. Anh không giải bài thay em, và em biết ơn vì điều đó. Mỗi lần em bí, anh đặt lại câu hỏi theo một hướng khác để em tự nhìn ra chỗ mình nghĩ sai. Những lần anh yêu cầu em giải thích lại thay đổi của mình trước khi cho lên hệ thống thật đã dạy em thói quen tự chất vấn công việc của chính mình, và đó là thứ em mang theo được lâu nhất.

Em cũng xin cảm ơn các anh chị trong nhóm phát triển sản phẩm khách sạn. Cảm ơn các anh chị đã kiên nhẫn với một người mới còn hỏi những câu rất cơ bản, đã chỉ cho em vì sao một đoạn mã trông vô hại lại có thể làm chậm cả hệ thống, và đã nhiều lần dừng việc của mình để xem lại mã của em.

Em xin cảm ơn các thầy cô Trường Đại học Công nghệ, Đại học Quốc gia Hà Nội. Những môn học tưởng như khô khan về cơ sở dữ liệu, cấu trúc dữ liệu và thuật toán, công nghệ phần mềm hoá ra lại chính là thứ em dùng đến hằng ngày. Có những buổi em ngồi tối ưu một truy vấn và nhận ra mình đang làm đúng thứ đã học trên lớp, chỉ khác là lần này số liệu là thật và người dùng cũng thật.

Cuối cùng, em xin cảm ơn gia đình và bạn bè đã ở bên em trong suốt kỳ thực tập này.

Hà Nội, tháng 9 năm 2026

Sinh viên

Trần Quang Thành

I. Giới thiệu chung

a. Giới thiệu công ty

KiotViet là thương hiệu phần mềm quản lý bán hàng do Công ty Cổ phần Công nghệ Citigo phát triển, ra đời với mục tiêu giúp những cửa hàng vừa và nhỏ ở Việt Nam quản lý việc bán hàng bằng phần mềm thay vì bằng sổ tay. Từ nhóm khách hàng ban đầu là các cửa hàng bán lẻ, công ty mở rộng dần sang nhà hàng, quán cà phê, cửa hàng làm đẹp và khách sạn; mỗi ngành có cách vận hành riêng nên được tách thành những dòng sản phẩm khác nhau dùng chung phần nền bên dưới.

Em thực tập tại nhóm phát triển sản phẩm cho ngành khách sạn. Sản phẩm phục vụ khách sạn, nhà nghỉ, homestay và resort, với người dùng cuối là lễ tân, thu ngân, buồng phòng, kế toán và chủ cơ sở lưu trú. Điểm đáng chú ý là sản phẩm không chỉ chạy ở Việt Nam mà còn được dùng ở một số nước khác trong khu vực, nên phần mềm phải nói được nhiều thứ tiếng và tính toán đúng theo múi giờ, đơn vị tiền tệ của từng nơi.

b. Giới thiệu công việc

Em tham gia nhóm với vai trò thực tập sinh phát triển phần mềm, làm cả phần giao diện và phần xử lý phía máy chủ, từ ngày 12 tháng 3 năm 2026 đến ngày 7 tháng 9 năm 2026.

Ngày đầu em hơi choáng. Sản phẩm không nằm trong một kho mã duy nhất mà chia thành chín kho mã riêng: màn hình cho lễ tân, màn hình cho người quản lý, dịch vụ xử lý đơn đặt phòng, dịch vụ sinh báo cáo, dịch vụ xuất nhập tệp dữ liệu, phần giao tiếp với ứng dụng di động và phần nền dùng chung. Tổng cộng có hàng trăm nghìn dòng mã đã tồn tại từ trước, do rất nhiều người viết qua nhiều năm. Em mất gần hai tuần đầu chỉ để trả lời được câu hỏi đơn giản nhất: khi lễ tân bấm một nút trên màn hình thì dữ liệu chạy qua những đâu.

Công việc của em chia thành bốn nhóm: sửa lỗi được báo về từ khách hàng đang dùng thật, làm tính năng mới theo yêu cầu, tối ưu hiệu năng ở những chỗ làm hệ thống chậm hoặc chặn nhau, và tham gia dựng phần quản lý phiên bản mới.

Bảng 1 tóm tắt phần đóng góp của em, số liệu lấy trực tiếp từ lịch sử mã nguồn. Con số dòng mã đã loại bỏ các tệp bản dịch ngôn ngữ, tệp cấu hình sinh tự động và tệp nhị phân, để phản ánh đúng phần mã do em thực sự viết.

c. Giới thiệu qua bài toán

Nghe tên thì phần mềm quản lý khách sạn có vẻ đơn giản: khách đến thì nhận phòng, khách đi thì trả phòng và thanh toán. Nhưng khi bắt tay vào làm em mới thấy phần khó không nằm ở chỗ lưu dữ liệu, mà ở chỗ cùng một dữ liệu phải trả lời đúng cho rất nhiều

Bảng 1. Tóm tắt đóng góp trong kỳ thực tập

Hạng mục	Số liệu
Thời gian thực tập	12/03/2026 đến 07/09/2026
Số kho mã đã tham gia	9
Số lần ghi mã, không tính lần gộp nhánh	698
Số đầu việc khác nhau đã xử lý	342
Số phiếu công việc đã hoàn thành	173
Số ngày có ghi mã	99
Số dòng mã thêm mới	34.994
Số dòng mã xoá bỏ	10.079
Số lượt sửa tệp	1.947
Số dòng mã kiểm thử tự động đã viết	1.919

câu hỏi khác nhau, ở nhiều nơi khác nhau, tại nhiều thời điểm khác nhau.

Lấy một ví dụ em gặp thật. Một phòng bị khoá lại để sửa chữa trong ba ngày. Việc này nghe rất nhỏ, nhưng ngay sau đó hàng loạt câu hỏi khác phải đổi câu trả lời: màn hình lễ tân phải không cho đặt phòng đó nữa, lịch phòng phải vẽ ra một khoảng bị chặn, báo cáo công suất phòng phải hiểu rằng phòng này đang không kinh doanh nên không được tính vào mẫu số, và nếu người quản lý xem lại số liệu tháng trước thì phòng đó lại phải được tính bình thường vì lúc ấy nó chưa bị khoá. Một thay đổi ở một chỗ kéo theo hệ quả ở năm sáu chỗ khác, và những chỗ đó do người khác viết từ trước.

Bài toán còn khó thêm vì ba lý do. Thứ nhất, hệ thống đang chạy thật nên không có chuyện làm lại từ đầu. Thứ hai, một khách hàng có thể có nhiều chi nhánh ở nhiều nơi, mỗi nơi một múi giờ và một đồng tiền, nên câu hỏi tưởng dễ như hôm nay bán được bao nhiêu cũng cần định nghĩa rõ hôm nay là theo giờ ở đâu. Thứ ba, phần mềm hiển thị được năm ngôn ngữ, nên mỗi chuỗi chữ, mỗi định dạng ngày và mỗi cách viết số đều phải xử lý cho cả năm. Trong bối cảnh đó, phần việc em nhận không phải là xây một hệ thống mới, mà là bổ sung và làm tốt hơn những phần cụ thể của một hệ thống đang sống.

II. Yêu cầu bài toán

a. Miêu tả chi tiết bài toán

Hệ thống phục vụ ba nhóm người dùng có nhu cầu rất khác nhau, và chính sự khác nhau đó tạo ra phần lớn độ khó.

Lễ tân và thu ngân làm việc ở quầy, thường vào lúc đông khách, nên cần thao tác nhanh, ít bước, thấy ngay tình trạng phòng. Màn hình của họ phải trả lời tức thì những câu như phòng nào đang trống, khách này đã trả tiền chưa, đơn này còn thiếu bao nhiêu. Nếu màn hình chậm một giây thì khách đứng chờ ở quầy sẽ thấy ngay.

Người quản lý và chủ cơ sở thì quan tâm con số tổng: doanh thu tháng này so với tháng trước, công suất phòng bao nhiêu phần trăm, nguồn khách nào mang lại nhiều tiền nhất. Câu hỏi của họ luôn là câu hỏi tổng hợp trên một lượng dữ liệu lớn, và điều họ khó tha thứ nhất là hai màn hình cùng nói về một chỉ tiêu nhưng cho ra hai con số khác nhau. Nhóm thứ ba là bộ phận kế toán và buồng phòng, với những nhu cầu rất cụ thể như xuất hoá đơn, đối chiếu sổ quỹ, bàn giao ca, theo dõi tình trạng dọn phòng.

Yêu cầu chung đặt lên hệ thống, theo cách em hiểu sau sáu tháng, gồm bốn điểm: số liệu phải nhất quán, nghĩa là cùng một chỉ tiêu thì mọi màn hình và mọi báo cáo phải cho cùng một kết quả; quyền hạn phải được tôn trọng, và việc chặn phải nằm ở phía máy chủ chứ không phải chỉ ẩn nút đi trên giao diện; hệ thống phải đúng theo bối cảnh địa phương gồm ngôn ngữ, múi giờ và đơn vị tiền tệ; và hệ thống phải đủ nhanh khi nhiều chi nhánh cùng dùng, một người bấm xuất tệp dữ liệu lớn thì không được làm những người còn lại phải chờ.



Hình 1. Các thành phần chính của hệ thống và quan hệ giữa chúng

Hình 1 là cách em vẽ lại hệ thống sau khi đã làm việc với nó một thời gian. Điều em muốn nhấn mạnh là các dịch vụ không tách rời nhau. Một tính năng thật gần như luôn phải đi qua nhiều hộp trong hình, và đó là lý do việc đọc hiểu phần người khác đã viết lại quan trọng hơn việc viết mã mới.

b. Phần bài toán em trực tiếp giải quyết

Toàn bộ sản phẩm là công việc của cả một nhóm nhiều người trong nhiều năm, nên em xin nói rõ phần nào là phần của mình. Em không tham gia thiết kế kiến trúc tổng thể, cũng không quyết định hướng đi của sản phẩm. Phần em nhận là những bài toán con đã

được xác định rõ đầu vào và đầu ra, còn cách giải thì để người làm tự quyết. Trong sáu tháng, phần việc của em rơi vào sáu hướng.

1. Làm mới tính năng khoá phòng để sửa chữa, đi từ bảng dữ liệu, quy tắc nghiệp vụ, đầu giao tiếp, cho tới giao diện, bản dịch ngôn ngữ và kiểm thử tự động.
2. Bổ sung hai chỉ tiêu thống kê mới về lượt khách nghỉ qua đêm và tổng số đêm nghỉ. Đây là bài toán mang tính thuật toán rõ nhất em gặp trong kỳ thực tập.
3. Tách một quyền hạn cũ thành hai quyền riêng biệt, kèm việc chuyển đổi dữ liệu phân quyền của toàn bộ khách hàng đang dùng sang cấu trúc mới.
4. Tối ưu hiệu năng: sửa những chỗ truy vấn cơ sở dữ liệu kém, những chỗ giữ khoá quá lâu làm người khác phải chờ, và những chỗ giao diện tính toán lại nhiều lần không cần thiết.
5. Dựng các màn hình cho bản quản lý mới, gồm màn hình tổng quan với các biểu đồ doanh thu, màn hình danh sách và chi tiết phòng, và phần sổ tay hướng dẫn tích hợp kênh bán phòng trực tuyến.
6. Xử lý phần đa ngôn ngữ và bối cảnh địa phương, gồm bổ sung tiếng Hàn, tiếng Trung, tiếng Lào cho nhiều màn hình, và sửa những lỗi định dạng ngày tháng, đơn vị tiền tệ, múi giờ.

c. Công việc của nhóm và công việc của em

Nhóm có nhiều vai trò: bộ phận phân tích nghiệp vụ nói chuyện với khách hàng và viết ra yêu cầu, bộ phận thiết kế giao diện dựng bản vẽ, người phát triển hiện thực yêu cầu, bộ phận kiểm thử xác nhận kết quả trước khi cho phát hành, bộ phận hỗ trợ khách hàng tiếp nhận phản hồi từ người dùng thật và chuyển ngược lại thành phiếu công việc, còn trưởng nhóm phân việc, xem lại mã và quyết định thứ tự ưu tiên.

Trong sơ đồ đó, em ở vị trí người phát triển. Việc của em bắt đầu khi nhận một phiếu và kết thúc khi thay đổi của em chạy được trên hệ thống thật mà không gây ra vấn đề mới. Ở giữa hai mốc đó, em tự làm các bước: đọc yêu cầu và hỏi lại cho rõ, đọc mã cũ để hiểu chỗ mình sắp chạm vào, chọn cách sửa, viết mã, tự kiểm tra, viết kiểm thử tự động cho những phần logic quan trọng, đưa mã lên để đồng nghiệp xem lại, sửa theo góp ý, rồi phối hợp với bộ phận kiểm thử cho đến khi phiếu được đóng.

Có một chi tiết em muốn nhắc vì nó thay đổi cách em làm việc. Ở hai tháng đầu, em nghĩ nhiệm vụ của mình là làm cho yêu cầu chạy được; về sau em hiểu nhiệm vụ thật là làm cho yêu cầu chạy được mà không làm hỏng thứ đang chạy. Điều thứ hai buộc em phải đi tìm những chỗ mình có thể vô tình ảnh hưởng, và đó thường là phần mất nhiều thời gian nhất của một phiếu.

III. Tóm tắt lý thuyết, giải pháp, thuật toán

a. Các lý thuyết và giải pháp liên quan

Phần này ghi lại những khái niệm em thực sự phải hiểu để làm được việc. Mỗi khái niệm đều gắn với một tình huống cụ thể đã buộc em tìm hiểu nó, và em sẽ nói rõ tình huống đó ở mục sau.

- **Phân tách theo tầng.** Tầng ngoài nhận yêu cầu và trả kết quả, tầng giữa chứa quy tắc nghiệp vụ, tầng trong nói chuyện với cơ sở dữ liệu. Lúc đầu em thấy rườm rà, nhưng khi phải thêm cùng một tính năng vào hai nơi khác nhau thì mới thấy lợi ích: quy tắc nằm riêng một tầng nên chỉ viết một lần, hai đầu giao tiếp cùng gọi vào đó.

- **Tách lệnh khỏi truy vấn.** Dịch vụ đặt phòng không sửa trực tiếp vào bản ghi mà sinh ra một lệnh mô tả ý định, các lệnh được xử lý tuần tự, rồi từ chuỗi lệnh đó hệ thống dựng nên một bản dữ liệu riêng dành cho việc đọc. Nhờ vậy lịch sử thay đổi được giữ đầy đủ và phần đọc tối ưu được riêng, nhưng người mới mất nhiều thời gian hơn để hiểu.

- **Chi phí của tầng trừu tượng khi truy cập dữ liệu.** Thư viện cho phép viết truy vấn bằng chính ngôn ngữ lập trình rồi tự dịch sang câu lệnh cơ sở dữ liệu. Rất tiện, nhưng câu lệnh gửi đi không phải thứ mình nhìn thấy khi đọc mã: một biểu thức lồng nhiều lớp, có nối bảng kiểu lấy cả phần thiếu, có ghép nhiều nguồn rồi lọc trùng, khi dịch ra có thể thành câu lệnh rất dài mà cơ sở dữ liệu không tối ưu nổi.

- **Khoá phân tán và thời gian sống của khoá.** Có những đoạn việc chỉ được phép một tiến trình làm tại một thời điểm. Rủi ro là nếu tiến trình đang giữ khoá bị dừng đột ngột thì khoá không bao giờ được nhả và mọi tiến trình khác chờ mãi, nên phải gắn cho khoá một thời gian sống để nó tự mất đi khi hết hạn.

- **Cơ chế phát hiện thay đổi của giao diện.** Mỗi lần thư viện kiểm tra dữ liệu đã đổi chưa thì mọi biểu thức đang hiển thị đều bị tính lại. Nếu ta viết một thuộc tính tính toán rồi đem hiển thị, với danh sách vài chục dòng mà mỗi biểu thức lại quét cả mảng để tính tổng, chi phí nhân lên rất nhanh và màn hình bị giật.

- **Gộp khoảng và giao khoảng.** Cho một tập khoảng thời gian có thể chồng lấn, hãy gộp những khoảng chồng lấn thành khoảng liền rồi tính phần giao với một khoảng quan sát cho trước. Bước gộp: sắp xếp theo thời điểm bắt đầu, đi lần lượt, nếu khoảng đang xét bắt đầu sau khi khoảng cuối trong kết quả đã kết thúc thì thêm mới, ngược lại thì nới điểm kết thúc ra cho đủ. Bước cắt: điểm bắt đầu lấy giá trị lớn hơn giữa nó và mốc đầu kỳ, điểm kết thúc lấy giá trị nhỏ hơn giữa nó và mốc

cuối kỳ, và nếu kết quả âm thì coi như bằng không.

- **Băm dữ liệu định danh.** Dữ liệu định danh của khách là dữ liệu cá nhân nên hệ thống lưu dạng đã băm, tức là biến chuỗi gốc thành một dãy ký tự có độ dài cố định theo một hàm một chiều. Hai chuỗi gốc giống nhau cho ra cùng một dãy, còn từ dãy đó thì không suy lại được chuỗi gốc, nhờ vậy so sánh được mà không cần đọc thông tin gốc.
- **Chuẩn hoá văn bản để tìm kiếm nhiều ngôn ngữ.** Người dùng gõ không giống hệt nội dung được lưu, nên phải chuẩn hoá cả hai phía trước khi so sánh rồi tách thành các từ và yêu cầu tiêu đề chứa tất cả các từ đó. Điểm cần lưu ý là không được định nghĩa chữ theo bảng chữ cái tiếng Anh, vì làm vậy sẽ xoá mất dấu tiếng Việt và toàn bộ chữ Hàn, chữ Trung.

b. Cách giải quyết của em

Tính năng khoá phòng: sinh lệnh theo sai khác trạng thái

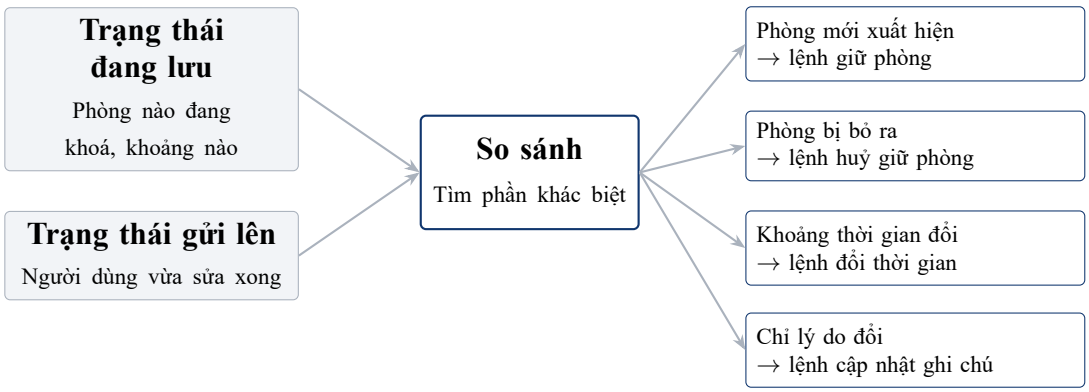
Đây là tính năng lớn nhất em làm. Yêu cầu nghe gọn: cho phép người quản lý khoá một hoặc nhiều phòng trong một khoảng thời gian kèm lý do, để phòng đó tạm không kinh doanh.

Phần khó xuất hiện ở thao tác sửa. Người dùng mở một lượt khoá đã có, rồi có thể đồng thời bỏ vài phòng, thêm vài phòng khác, đổi khoảng thời gian và sửa lý do. Cách đơn giản nhất là huỷ sạch lượt khoá cũ rồi tạo lại. Em đã nghĩ tới cách này trước vì nó dễ viết nhất, nhưng nó có hai điểm dở: lịch sử thay đổi bị mất nên người quản lý không còn biết ai đã sửa gì, và những phòng người dùng không hề chạm tới cũng bị huỷ rồi tạo lại, tạo ra một loạt biến động vô nghĩa và tăng nguy cơ đụng vào các đơn khác trong khoảng thời gian đó.

Cách em chọn là so sánh trạng thái hiện tại với trạng thái người dùng vừa gửi lên, rồi chỉ sinh đúng những lệnh phản ánh phần khác biệt. Em viết một thành phần riêng chuyên làm việc này, tách hoàn toàn khỏi phần nhận yêu cầu và phần lưu dữ liệu, nên nó không phụ thuộc cơ sở dữ liệu và kiểm thử được độc lập.

Hình 2 mô tả ý tưởng. Nếu người dùng chỉ sửa lý do, hệ thống chỉ sinh một lệnh cập nhật ghi chú; nếu bỏ toàn bộ phòng mà không chọn phòng mới thì bị chặn lại ngay. Em cũng tách riêng các quy tắc kiểm tra dữ liệu vào một chỗ: loại khoá phải nằm trong danh sách cho phép, lý do không dài quá giới hạn, phải chọn ít nhất một phòng, không được chọn trùng một phòng hai lần, thời điểm kết thúc phải sau thời điểm bắt đầu. Việc gom các quy tắc này giúp thông báo lỗi nhất quán, và quan trọng hơn là mọi đường vào hệ thống đều chịu cùng một bộ kiểm tra.

Ở phía giao diện, em làm một hộp thoại riêng cho việc khoá phòng, gắn vào màn



Hình 2. Chỉ sinh lệnh cho phần thực sự thay đổi thay vì huỷ và tạo lại toàn bộ

hình lịch phòng để người dùng thấy khoảng bị khoá ngay trên lịch, và bổ sung phần kiểm tra khi đặt phòng. Toàn bộ chuỗi chữ hiển thị được khai báo cho cả ba ngôn ngữ ngay trong cùng lần làm, vì em đã học được rằng để lại việc dịch cho sau thì gần như chắc chắn sẽ quên. Nhưng phần em tâm đắc nhất lại không phải phần mã chính mà là phần đi tìm hậu quả. Sau khi tính năng chạy được, em tự hỏi những chỗ nào khác đang đếm phòng mà chưa biết đến khái niệm phòng bị khoá, và thấy các phép tính công suất phòng vẫn tính phòng bị khoá vào phần có thể bán được, khiến công suất thấp hơn thực tế và người quản lý bị đánh giá sai. Em đã sửa các phép tính này để loại phòng đang khoá ra khỏi mẫu số. Việc này không có trong yêu cầu, không ai nhắc em, và nếu em không làm thì lỗi sẽ chỉ lộ ra sau vài tuần khi có người thắc mắc về con số.

Thống kê lượt khách nghỉ qua đêm và tổng số đêm nghỉ

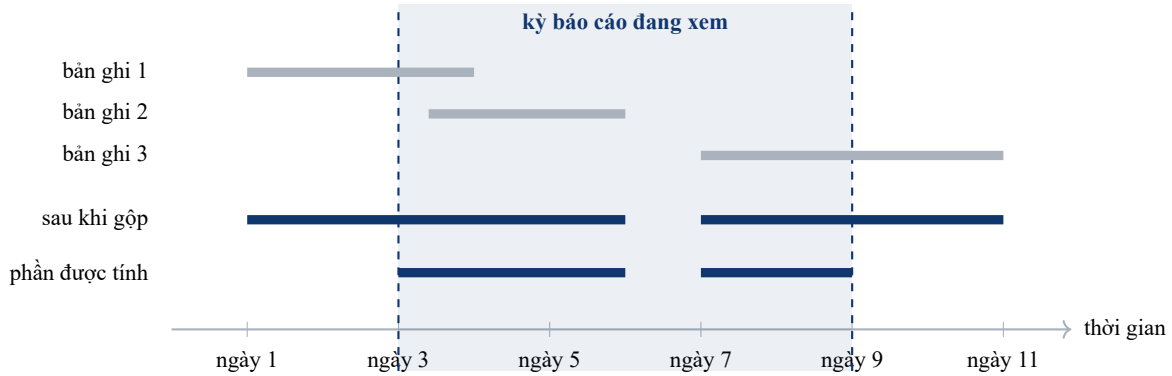
Yêu cầu là bổ sung hai chỉ tiêu vào báo cáo phòng cho một khoảng thời gian tuỳ chọn. Nghe qua tưởng chỉ cần đếm, nhưng khi phân tích em thấy có ba vấn đề tách biệt.

Vấn đề thứ nhất là chồng lấn. Cùng một người có thể có nhiều bản ghi lưu trú chồng lên nhau hoặc nối tiếp nhau, nên cộng thẳng số đêm của từng bản ghi thì những ngày bị tính hai lần sẽ làm con số phồng lên. Em giải quyết bằng thuật toán gộp khoảng đã nói ở trên.

Vấn đề thứ hai là biên của kỳ báo cáo. Một lượt lưu trú có thể bắt đầu trước ngày đầu kỳ và kết thúc sau ngày cuối kỳ. Em tách hai chỉ tiêu ra xử lý khác nhau: số lượt chỉ được tính nếu ngày bắt đầu nằm trong kỳ đang xem, vì lượt là một sự kiện xảy ra tại một thời điểm; còn số đêm thì phải cắt theo kỳ, chỉ đếm phần đêm thực sự nằm trong kỳ, kể cả khi lượt đó bắt đầu từ trước.

Hình 3 minh hoạ cả hai bước. Ba bản ghi ban đầu chồng lấn nhau, sau khi gộp còn hai khoảng rời, và khi cắt theo kỳ thì phần nằm ngoài hai đường kẻ đứt bị bỏ đi. Bản ghi 1 bắt đầu trước kỳ nên không được tính là một lượt, nhưng những đêm của nó nằm trong kỳ vẫn được cộng vào tổng số đêm.

Vấn đề thứ ba là nhận diện cùng một người. Em dùng dữ liệu định danh đã băm



Hình 3. Gộp các khoảng lưu trú chồng lấn rồi cắt theo biên của kỳ báo cáo

làm chìa khoá gom nhóm, và ở đây em gặp cái bẫy đáng nhớ nhất của cả kỳ thực tập. Với những khách không khai giấy tờ, chuỗi định danh là chuỗi rỗng. Nhưng chuỗi rỗng khi đưa qua hàm băm vẫn cho ra một dãy ký tự cố định, và dãy đó giống nhau với mọi khách không khai giấy tờ. Nghĩa là nếu gom nhóm một cách hồn nhiên, toàn bộ khách không khai giấy tờ trong kỳ sẽ bị coi là cùng một người duy nhất, báo cáo sẽ báo một lượt trong khi thực tế có thể là hàng trăm lượt.

Điều đáng lo là lỗi này rất khó bị phát hiện khi kiểm thử thủ công, vì với dữ liệu mẫu nhỏ chỉ có một hai khách chưa khai giấy tờ thì con số vẫn trông hợp lý. Em xử lý bằng cách nhận biết đúng dãy băm của chuỗi rỗng và coi nó là trường hợp không có định danh, khi đó chuyển sang dùng mã bản ghi làm chìa khoá gom nhóm để mỗi khách vẫn được đếm riêng, kèm một trường hợp kiểm thử mang tên nói rõ ý định để người sau có sửa lại phần này thì cũng không vô tình làm lỗi quay lại.

Em viết toàn bộ phần tính toán này thành một lớp thuần, chỉ nhận dữ liệu vào và trả kết quả ra, không chạm tới cơ sở dữ liệu, nhờ vậy kiểm thử được đầy đủ mười ba trường hợp gồm cả những trường hợp biên khó dựng bằng tay. Trong đó có một trường hợp em thấy hữu ích hơn cả: em kiểm tra rằng tổng của hai kỳ liền nhau phải bằng đúng kết quả tính một lần cho cả khoảng gộp lại. Đây không phải kiểm tra một giá trị cụ thể mà là kiểm tra một tính chất phải luôn đúng, và nó bắt được những sai sót ở biên mà việc thử từng số riêng lẻ dễ bỏ qua. Cũng cần nói thêm rằng thông tin định danh của khách được lưu ở kho dữ liệu riêng và được mã hoá, nên phần tính toán của em phải đọc từ hai nguồn rồi ghép lại theo mã bản ghi. Việc này bất tiện hơn là để tất cả cùng một chỗ, nhưng em coi đây là lần đầu mình được làm việc với dữ liệu cá nhân theo đúng cách.

Tối ưu việc xuất dữ liệu và hai lỗi hiệu năng đáng nhớ

Chức năng xuất tệp sổ quỹ bị báo chậm. Em đọc thì thấy phần lấy tên phòng cho mỗi dòng dữ liệu được viết bằng bốn năm biểu thức truy vấn khác nhau, mỗi biểu thức nối vài bảng theo kiểu lấy cả phần thiếu, rồi ghép lại và lọc trùng; ở giữa còn một câu kiểm tra xem có dữ liệu hay không, và chính câu đó cũng là một lượt đi tới cơ sở dữ liệu. Nhưng

chỗ tệ nhất không nằm ở truy vấn: sau khi lấy được dữ liệu về, mã cũ đi qua từng dòng của tệp kết quả, và với mỗi dòng lại quét lại toàn bộ danh sách vừa lấy về. Nếu tệp có n dòng và danh sách có m phần tử thì số phép so sánh là n nhân m, và với tệp vài nghìn dòng thì con số này đủ lớn để người dùng cảm nhận được.

Em làm lại theo ba bước: nếu không có dòng nào cần xử lý thì thoát ra ngay; thay toàn bộ chuỗi biểu thức truy vấn bằng một câu lệnh viết sẵn phía cơ sở dữ liệu, gọi một lần và nhận về đúng dữ liệu cần; và thay vì quét lại danh sách cho từng dòng thì gom dữ liệu một lần thành một bảng tra theo chìa khoá. Số phép tính giảm từ n nhân m xuống còn khoảng n cộng m, và phần mã sau khi làm lại ngắn hơn phần mã cũ khoảng hai phần ba. Điều này dạy em rằng mã nhanh hơn không đồng nghĩa với mã phức tạp hơn: ở đây cách viết nhanh hơn cũng chính là cách viết dễ đọc hơn.

Một phiếu khác báo rằng khi có người xuất tệp hoá đơn kèm chi tiết thì các thao tác khác trên cùng dữ liệu bị chậm hẳn, tức là hiện tượng chặn nhau ở tầng cơ sở dữ liệu. Em không đi sửa cách quản lý giao dịch vì đó là phần chung, chạm vào sẽ ảnh hưởng rộng, mà đi tìm câu hỏi khác: tại sao lượt đọc này lại nặng như vậy. Đọc mã thì thấy phần khai báo dữ liệu cần lấy kèm đang yêu cầu ba nhóm dữ liệu mà tệp kết quả không hề dùng tới, và trong danh sách còn có phần tử bị lặp; mỗi nhóm lấy kèm làm câu lệnh phải nối thêm bảng, khiến phạm vi khoá rộng ra và thời gian giữ khoá dài ra. Em bỏ ba nhóm không dùng và lọc trùng danh sách, thay đổi cuối cùng chỉ vài dòng. Nếu lao vào sửa cơ chế giao dịch thì vừa mất thời gian, vừa rủi ro, mà vẫn không giải quyết được nguyên nhân thật.

Lỗi thứ hai em tự tìm ra chứ không ai báo: khoá phân tán được tạo ra mà không đặt thời gian sống, nghĩa là nếu tiến trình giữ khoá dừng bất thường thì khoá sẽ nằm lại vĩnh viễn và mọi tiến trình sau đó đều bị chặn. Em bổ sung thời gian sống cho khoá, chỉ năm dòng và không thêm tính năng nào cho người dùng, nhưng đây là lần đầu em tự tìm ra một rủi ro chưa xảy ra thay vì chờ nó xảy ra rồi mới đi sửa.

Tập trung quy tắc phân quyền vào một chỗ

Yêu cầu là tách một quyền hạn cũ về việc huỷ đơn thành hai quyền riêng, phân biệt theo tình trạng của khách. Phần khó có hai mặt.

Mặt thứ nhất là dữ liệu. Toàn bộ khách hàng đang dùng đã có sẵn quyền cũ gán cho các vai trò của họ, nên phải chuyển đổi dữ liệu phân quyền sang cấu trúc mới sao cho không ai bị mất quyền đang có. Em viết một tập lệnh chuyển đổi, và cùng với nó viết luôn một tập lệnh đưa dữ liệu về trạng thái cũ. Ban đầu em thấy việc viết phần quay lại là dư thừa vì mình tin tập lệnh của mình chạy đúng, nhưng anh hướng dẫn nói một câu em nhớ mãi, đại ý là thứ cần chuẩn bị không phải cho trường hợp mình sai, mà cho trường hợp mình không có thời gian tìm hiểu vì sao mình sai.

Mặt thứ hai là giao diện. Việc kiểm tra quyền cần xuất hiện ở nhiều chỗ trên màn hình lễ tân. Cách nhanh nhất là rải các câu điều kiện vào từng chỗ, nhưng em không làm vậy vì đã thấy hậu quả của lối viết đó: mỗi lần quy tắc đổi là phải đi tìm lại tất cả những chỗ đã rải, và chỉ cần sót một chỗ là hành vi không còn nhất quán. Em gom toàn bộ quy tắc vào một thành phần chuyên trách về quyền huỷ đơn, các thành phần hiển thị chỉ hỏi nó một câu duy nhất là người dùng hiện tại có được huỷ đơn này hay không.

Cần nói thêm rằng việc ẩn hay hiện nút trên giao diện không phải là bảo mật, nó chỉ là trải nghiệm người dùng. Việc chặn thật phải nằm ở phía máy chủ, vì người dùng hoàn toàn có thể gọi trực tiếp vào đầu giao tiếp mà không đi qua giao diện.

Giữ cho giao diện nhẹ và tách logic khỏi phần hiển thị

Nhóm em có quy ước không dùng thuộc tính tính toán đem hiển thị, và thay bằng hai cách. Cách thứ nhất là dùng một biến lưu kết quả, gán lại tại đúng những thời điểm dữ liệu thay đổi, còn phần hiển thị chỉ đọc biến; em đã viết theo cách này cho thông tin lượt khoá phòng đang xem, tiêu đề hộp thoại theo chế độ đang mở, và bảng tổng kết các dòng thanh toán. Cách thứ hai dùng khi giá trị phụ thuộc tham số của từng dòng trong danh sách, lúc đó nhóm có sẵn một bộ chuyển đổi thuần chỉ chạy lại khi tham số thay đổi. Phần này thay đổi cách em nhìn về hiệu năng: trước đây em nghĩ tối ưu là việc làm sau, giờ em thấy hiệu năng là một thói quen viết mã, và một quy ước đơn giản được cả nhóm tuân theo có tác dụng lớn hơn nhiều so với việc để một người ngồi tối ưu về sau.

Cùng tinh thần đó, khi làm ô tìm kiếm cho phần sổ tay hỗ trợ tích hợp kênh bán phòng trực tuyến, em tách logic tìm kiếm ra khỏi phần hiển thị thành một lớp tiện ích thuần chỉ gồm ba việc: chuẩn hoá chuỗi, tách chuỗi thành các từ, và đối chiếu. Khi chuẩn hoá, em khai báo chữ và số theo thuộc tính ký tự thay vì liệt kê bảng chữ cái, để dấu tiếng Việt và các hệ chữ khác không bị xoá mất; khi đối chiếu, em yêu cầu tiêu đề chứa tất cả các từ người dùng gõ nhưng không cần đúng thứ tự.

Dựng lại biểu đồ doanh thu và một lỗi lặp lại ba lần

Ở bản quản lý viết lại, em nhận việc dựng màn hình tổng quan với các biểu đồ về doanh thu. Thành phần cũ là một tệp dài trộn lẫn phần lấy dữ liệu, phần tính toán và phần vẽ, nên em chọn viết lại và tách thành ba tệp riêng, đồng thời chuyển việc gọi dữ liệu ra tầng riêng theo đúng cách tổ chức của dự án.

Trong lúc làm, em nhận được ba phiếu lỗi khác nhau, do ba người báo, ở ba chỗ khác nhau trên cùng màn hình tổng quan: bỏ chọn một chi nhánh nhưng doanh thu của chi nhánh đó vẫn được tính vào tổng, biểu đồ hình tròn không tính lại, và khối tổng tiền phòng cũng không tính lại. Nếu em sửa từng phiếu một thì sẽ có ba chỗ sửa riêng và cả ba lần đều là sửa đúng theo yêu cầu. Nhưng khi đọc kỹ em thấy chúng chỉ là một vấn đề:

các thành phần con đang truyền cho nhau danh sách mã chi nhánh, và mỗi thành phần lại tự giữ một bản sao rồi tự xử lý theo cách riêng, nên khi người dùng bỏ chọn một chi nhánh thì có thành phần nhận được thông tin cập nhật, có thành phần thì không. Em sửa ở gốc: thay vì truyền danh sách mã, các thành phần cùng nhận và cùng dùng một đối tượng chi nhánh thống nhất. Sau lần sửa đó cả ba phiếu đều hết, và loại lỗi này không quay lại ở những biểu đồ em thêm sau.

Bài học ở đây là một trong những bài học em thấy hữu ích nhất cả kỳ. Khi cùng một hiện tượng xuất hiện ở ba chỗ, rất có thể đó không phải ba lỗi mà là một vấn đề về thiết kế: sửa từng chỗ sẽ đóng được phiếu nhưng không giải quyết được nguyên nhân, và chỗ thứ tư sẽ xuất hiện vào lúc ta không mong đợi nhất.

c. Liên hệ và so sánh với các cách đã có

Về tính năng khoá phòng. Cách đơn giản là huỷ hết rồi tạo lại: ít mã, dễ hiểu, và nếu tính năng chỉ dùng nội bộ thì hoàn toàn chấp nhận được. Cách em chọn tốn công hơn và cần nhiều trường hợp kiểm thử hơn, nên cách nào tốt hơn phụ thuộc vào bối cảnh. Ở đây có ba yếu tố khiến cách thứ hai xứng đáng: lịch sử thay đổi là thứ người quản lý thực sự cần, phòng và khoảng thời gian có thể đụng tới các đơn khác nên càng ít biến động càng an toàn, và tính năng này sẽ được nhiều khách hàng dùng lâu dài. Nếu thiếu ba yếu tố đó thì cách đơn giản mới là cách đúng. Em ghi lại nhận xét này vì trước kỳ thực tập em hay mặc định rằng giải pháp phức tạp hơn là giải pháp tốt hơn.

Về nơi đặt phần tính toán thống kê. Đây là quyết định em làm sai trước rồi sửa lại. Ban đầu em hiện thực phần thống kê ở dịch vụ quản lý, vì báo cáo được xem từ màn hình quản lý nên đặt ở đó có vẻ tự nhiên. Ba ngày sau, khi ngồi xem lại cùng anh hướng dẫn, em nhận ra chỗ đặt như vậy không đúng: dữ liệu lưu trú thuộc về dịch vụ đặt phòng, và để dịch vụ quản lý tự đọc rồi tự tính sẽ dẫn tới hai chỗ trong hệ thống cùng định nghĩa một chỉ tiêu, đến lúc quy tắc tính đổi thì chỉ cần sửa một bên là hai màn hình cho ra hai con số khác nhau. Em đã huỷ bỏ toàn bộ phần đã làm và hiện thực lại ở dịch vụ đặt phòng, nơi sở hữu dữ liệu. Việc bỏ đi công sức mấy ngày không dễ chịu, nhưng em học được một nguyên tắc rõ ràng: phần tính toán nên nằm ở nơi sở hữu dữ liệu, còn các nơi khác thì gọi vào chứ không tự tính lại.

Về phạm vi ảnh hưởng của một thay đổi. Báo cáo hiển thị sai định dạng ngày khi dùng ngôn ngữ Hàn. Em đọc mã và thấy phần xử lý ngày có nhiều đoạn viết riêng cho từng ngôn ngữ, nằm rải ở nhiều tệp và trông rất chắp vá, nên em viết lại phần đó, bỏ đi khoảng một trăm năm mươi dòng ở sáu tệp khác nhau. Nhìn lại thì đó là một thay đổi rất rộng cho một lỗi rất hẹp. Ba mươi phút sau em tự huỷ bỏ thay đổi của mình, vì không có cách nào tự tin rằng sáu tệp đó không được dùng bởi những báo cáo khác mà em chưa hề mở ra xem. Sau đó em làm lại theo hướng hẹp, chỉ sửa đúng chỗ sinh ra lỗi

và bổ sung một hàm nhỏ để phần đọc ngày tháng chịu được nhiều dạng đầu vào hơn, gói trong một tệp và khoảng hai mươi dòng. Bài học không phải là đừng dọn mã, mà là dọn mã phải là một việc riêng có kế hoạch riêng và có kiểm thử che phủ trước, không nên gộp vào một phiếu sửa lỗi đang cần phát hành gấp.

IV. Mô tả phần mềm cài đặt

a. Phạm vi trình bày

Theo yêu cầu của báo cáo, phần này không mô tả cách cài đặt các ứng dụng phụ trợ như trình soạn mã hay công cụ xem cơ sở dữ liệu. Em cũng xin nói rõ rằng phần mềm này là sản phẩm thương mại đang được khai thác, nên em chỉ trình bày ở mức cách thức tổ chức và quy trình, không nêu địa chỉ máy chủ, thông số cấu hình, chuỗi kết nối hay bất kỳ chi tiết nào có thể ảnh hưởng tới hệ thống thật và tới dữ liệu của khách hàng.

b. Môi trường và công cụ cần có

Phần giao diện dùng Angular với TypeScript, trong đó màn hình lễ tân chạy trên Angular 9 còn màn hình quản lý viết lại trên Angular 18. Phần máy chủ dùng .NET và C#, truy cập dữ liệu qua Entity Framework, lưu dữ liệu trên SQL Server, dùng Redis làm bộ đệm và khoá phân tán, và các dịch vụ mới được đóng gói bằng Docker. Việc dựng mã và chạy kiểm thử được tự động hoá trên một máy chủ tích hợp liên tục. Về kiểm thử, phần máy chủ dùng xUnit còn phần giao diện dùng Jasmine cùng Karma.

Điểm đáng lưu ý là hai phiên bản Angular cách nhau khá xa, phản ánh thực tế của một sản phẩm đã sống nhiều năm: phần cũ vẫn phải chạy và vẫn phải sửa, trong khi phần mới được dựng song song. Trong nhiều tuần em làm trên cả hai, và việc chuyển qua lại giữa hai lối viết của cùng một thư viện buộc em phân biệt được đâu là khái niệm cốt lõi của công cụ, đâu chỉ là cách viết của một phiên bản.

c. Cách tổ chức mã nguồn và cách dựng phần mềm

Cả hai phía đều được tổ chức theo tầng, và cách đặt tên thư mục nói lên thứ tự các tầng. Ở phía máy chủ, một dịch vụ chia thành các nhóm: nhóm dùng chung chứa tiện ích và hằng số, nhóm trình bày chứa đầu giao tiếp và mô tả dữ liệu vào ra, nhóm ứng dụng chứa quy tắc nghiệp vụ cùng phần xử lý lệnh và truy vấn, nhóm hạ tầng chứa phần nói chuyện với cơ sở dữ liệu, và một nhóm riêng cho kiểm thử. Cách đánh số thứ tự vào tên thư mục nhắc người đọc chiều phụ thuộc: nhóm bên ngoài được biết về nhóm bên trong, ngược lại thì không. Phía giao diện có cấu trúc tương tự. Khi làm tính năng khoá phòng, em phải thêm tệp vào gần như tất cả các thư mục này, và chính lần đó em hiểu vì sao mỗi tầng lại tồn tại.

Về cách dựng, phần giao diện cần cài các gói phụ thuộc theo bản khai báo đã khoá phiên bản, chọn tệp cấu hình môi trường, rồi chạy lệnh dựng để sinh bộ tệp tĩnh đem đặt lên máy chủ web; riêng màn hình lễ tân còn có bản đóng gói chạy như ứng dụng trên máy tính tại quầy, nên khi sửa những phần liên quan tới in ấn hay thiết bị ngoại vi thì phải thử trên cả hai dạng. Phần máy chủ thì phục hồi các gói phụ thuộc, dựng bộ giải pháp, chạy toàn bộ kiểm thử tự động, rồi xuất bản.

d. Cách đưa thay đổi cơ sở dữ liệu

Đây là phần em cẩn thận nhất, vì mã nguồn sai thì có thể đưa lại bản cũ, còn dữ liệu bị sửa sai thì không phải lúc nào cũng lấy lại được. Các thay đổi về cấu trúc dữ liệu như thêm bảng, thêm cột hay sửa câu lệnh sẵn được quản lý trong một dự án riêng, để mọi thay đổi đều có lịch sử và đều đi qua bước xem lại. Các thay đổi về nội dung dữ liệu được viết thành tập lệnh riêng, và với mỗi tập lệnh chuyển đổi em luôn viết kèm một tập lệnh đưa dữ liệu về trạng thái trước đó; em đã làm việc này cho cả lần tách quyền huỷ đơn và lần bổ sung dữ liệu về quốc gia, múi giờ, đơn vị tiền tệ. Thói quen này em học được từ nhóm chứ không phải tự nghĩ ra, và đến giờ em thấy đó là một trong những thói quen giá trị nhất mình mang về.

e. Chuỗi môi trường và quy trình phát hành

Một thay đổi đi qua năm chặng. Em làm và tự thử trên nhánh riêng của mình, đồng nghiệp xem lại mã, rồi mã được gộp vào môi trường phát triển chung nơi hệ thống tự dựng và tự chạy kiểm thử. Tiếp đó mã lên môi trường kiểm thử để bộ phận kiểm thử xác nhận, rồi vào nhánh chuẩn bị phát hành để chốt phạm vi và chuẩn bị các tập lệnh dữ liệu, và cuối cùng lên hệ thống thật theo lịch, kèm việc theo dõi sau phát hành.

Điều em muốn nhấn mạnh là chặng thứ ba có một đường quay lại: nếu bộ phận kiểm thử còn tìm ra lỗi thì phiếu về lại nhánh riêng, và trong sáu tháng khá nhiều phiếu của em đã đi theo đường đó ít nhất một lần. Ban đầu mỗi lần bị trả lại em thấy khá nản, về sau em hiểu rằng một phiếu bị trả lại ở môi trường kiểm thử thì rẻ hơn rất nhiều so với cùng lỗi đó bị phát hiện bởi khách hàng.

Hai điểm khác trong quy trình em thấy đáng ghi lại. Thứ nhất, tính năng mới không được bật cho tất cả khách hàng cùng lúc mà bật dần theo từng nhóm, và việc chuyển khách hàng từ bản quản lý cũ sang bản mới cũng làm theo từng bước có mốc thời gian rõ ràng, em tham gia phần này ở giai đoạn cuối kỳ thực tập. Thứ hai, khi phát hành phiên bản mới của phần giao diện thì trình duyệt của người dùng vẫn giữ bản cũ trong bộ đệm nên họ mở lên và không thấy thay đổi. Đây là loại vấn đề em hoàn toàn không nghĩ tới trước khi đi thực tập, vì khi tự làm dự án ở nhà thì chỉ cần tự xoá bộ đệm là xong.

f. Kiểm thử tự động trong quy trình

Trong sáu tháng em viết khoảng một nghìn chín trăm dòng kiểm thử tự động, phân bố trên bảy tệp, gồm năm tệp cho phần máy chủ và hai tệp cho phần giao diện.

Ở phía máy chủ, các tệp kiểm thử tập trung vào những chỗ có logic thật: phần sinh lệnh cho tính năng khoá phòng, các quy tắc kiểm tra dữ liệu của tính năng đó, phần thông báo theo ngôn ngữ, và phần tính toán thống kê. Em đặt tên các trường hợp kiểm thử theo dạng mô tả tình huống và kết quả mong đợi, ví dụ bỏ hết phòng mà không chọn phòng mới thì phải bị chặn, nên danh sách kết quả chạy kiểm thử đọc lên gần như một bản mô tả hành vi.

Ở phía giao diện, em viết kiểm thử cho phần kiểm tra dữ liệu khi đặt phòng, vì đó là chỗ nhiều quy tắc chồng nhau: phòng đang bị khoá, phòng đang có khách chưa trả, và khoảng thời gian chờ dọn phòng sau khi khách trước rời đi. Trong đó có một trường hợp em đánh giá cao nhất: vẫn phải chặn những phòng đang có khách chưa trả, không phụ thuộc khoảng thời gian. Nó không kiểm tra tính năng em vừa làm, mà kiểm tra rằng tính năng cũ vẫn còn nguyên. Trước đây em nghĩ viết kiểm thử là để chứng minh mã của mình chạy đúng; bây giờ em nghĩ mục đích quan trọng hơn là để người sửa tiếp sau em biết ngay khi họ làm hỏng một hành vi mà họ không biết là có tồn tại.

V. Kết quả đạt được, hướng phát triển

a. Kỹ năng và kiến thức thu thập được

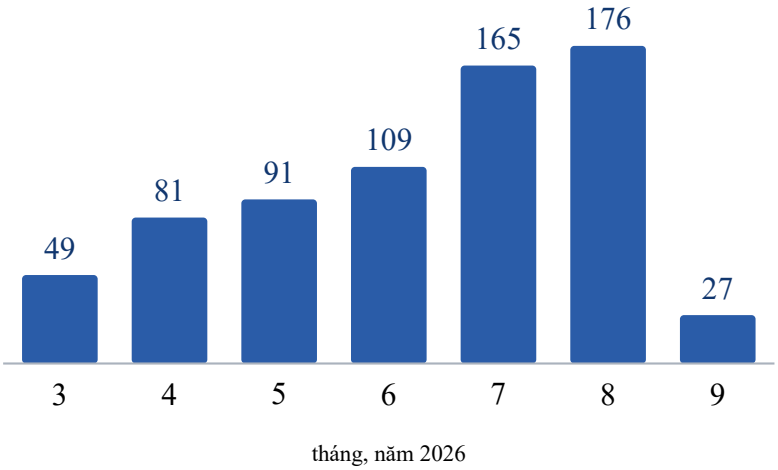
Những việc đã hoàn thành

Trong sáu tháng, em hoàn thành 173 phiếu công việc, tương ứng 342 đầu việc khác nhau, trải trên chín kho mã của sản phẩm. Những phần đáng kể nhất gồm:

- Tính năng khoá phòng để sửa chữa, làm trọn vẹn từ bảng dữ liệu, quy tắc nghiệp vụ, đầu giao tiếp cho tới giao diện, bản dịch ba ngôn ngữ và kiểm thử tự động, kèm việc sửa các phép tính công suất phòng để phòng đang khoá không bị tính sai.
- Hai chỉ tiêu thống kê mới về lượt khách nghỉ qua đêm và tổng số đêm nghỉ, với phần tính toán tách thành một lớp thuần và có mười ba trường hợp kiểm thử.
- Việc tách một quyền hạn cũ thành hai quyền riêng, gồm cả tập lệnh chuyển đổi dữ liệu cho toàn bộ khách hàng đang dùng và tập lệnh đưa về trạng thái trước đó.
- Bốn hạng mục tối ưu hiệu năng: giảm số lượt truy vấn và hạ chi phí tính toán khi xuất tệp sổ quỹ, chặn hiện tượng người dùng phải chờ nhau khi xuất tệp hoá đơn, đặt thời gian sống cho khoá phân tán, và áp dụng quy ước tránh tính toán lại trên

giao diện.

- Các màn hình cho bản quản lý viết lại, gồm màn hình tổng quan với các biểu đồ doanh thu, màn hình danh sách và chi tiết phòng, phần sổ tay hướng dẫn tích hợp kênh bán phòng trực tuyến kèm ô tìm kiếm, và việc tái cấu trúc bảng tổng kết của phiếu bàn giao ca.
- Phần đa ngôn ngữ và bối cảnh địa phương, gồm bổ sung tiếng Hàn, tiếng Trung, tiếng Lào cho nhiều màn hình và sửa các lỗi định dạng ngày tháng, đơn vị tiền tệ, múi giờ. Cùng với đó là khoảng một nghìn chín trăm dòng kiểm thử tự động ở cả hai phía.



Hình 4. Số lần ghi mã theo từng tháng trong kỳ thực tập. Tháng 9 chỉ gồm 7 ngày đầu nên cột thấp hơn hẳn.

Hình 4 nói về kỳ thực tập của em chính xác hơn mọi lời tự nhận xét. Tháng đầu em ghi mã ít nhất, không phải vì làm ít mà vì phần lớn thời gian ngồi đọc; con số tăng dần và cao nhất vào tháng 7 và tháng 8, đúng giai đoạn em đã đủ quen để tự nhận những phiếu lớn hơn. Nhưng cách hiểu đúng không phải là em làm việc hiệu quả hơn theo thời gian, mà là thời gian đọc hiểu ở giai đoạn đầu không mất đi, nó chuyển thành khả năng làm nhanh ở giai đoạn sau.

Kiến thức kỹ thuật

Về cơ sở dữ liệu, em học được cách đọc để hiểu một câu truy vấn thật sự làm gì thay vì chỉ đọc mã sinh ra nó, và cách nhận ra dấu hiệu của một truy vấn tồi như nối quá nhiều bảng, lọc trùng trên tập lớn, hay lấy kèm dữ liệu không dùng. Em cũng lần đầu viết câu lệnh sẵn phía cơ sở dữ liệu cho một chỗ cần hiệu năng, và lần đầu viết tập lệnh chuyển đổi dữ liệu cho hệ thống đang chạy thật.

Về hiệu năng, em hiểu rằng phần lớn vấn đề không nằm ở thuật toán tinh vi mà ở những việc rất cơ bản: làm nhiều lần một việc chỉ cần làm một lần, mang về nhiều dữ

liệu hơn mức cần dùng, và tìm kiếm tuần tự trong khi có thể tra theo chìa khoá. Trong bốn hạng mục tối ưu em làm, không hạng mục nào cần đến một thuật toán khó.

Về kiến trúc, em chuyển từ chỗ thấy việc chia tầng là rườm rà sang chỗ hiểu nó giải quyết vấn đề gì, và học được một nguyên tắc em tin sẽ theo mình lâu: phần tính toán nên nằm ở nơi sở hữu dữ liệu, và cùng một chỉ tiêu thì chỉ nên được định nghĩa ở đúng một chỗ. Về kiểm thử, em cũng thay đổi hẳn quan niệm: trước đây em coi nó là việc phải làm để đủ thủ tục, giờ em coi nó là một bản mô tả hành vi mà người sau đọc được, và một tấm lưới bảo vệ những hành vi cũ. Về xử lý nhiều ngôn ngữ và nhiều vùng, em nhận ra đây không phải việc thay chuỗi chữ như em từng nghĩ, mà gồm cả định dạng ngày tháng, cách viết số, ký hiệu tiền tệ, và quan trọng nhất là múi giờ: câu hỏi hôm nay bán được bao nhiêu là câu hỏi không có nghĩa cho đến khi xác định được hôm nay là theo giờ ở đâu.

Kỹ năng làm việc

Có những thứ em học được mà không liên quan đến kỹ thuật, và em nghĩ chúng quan trọng không kém.

Thứ nhất là biết hỏi. Tháng đầu em hay ngồi tự mò hai ba giờ vì sợ hỏi thì bị đánh giá là không biết gì. Sau vài lần được nhắc, em đổi cách: tự tìm khoảng ba mươi phút, nếu vẫn bí thì hỏi, nhưng hỏi kèm những gì mình đã thử và đang nghi ngờ ở đâu.

Thứ hai là viết mô tả cho thay đổi của mình. Đầu kỳ em viết những dòng như sửa lỗi hoặc cập nhật, và chính em hai tuần sau đó không hiểu mình đã sửa gì; về sau em viết rõ mã phiếu và nội dung thay đổi, mất thêm ba mươi giây nhưng tiết kiệm rất nhiều thời gian cho người phải lần lại.

Thứ ba là làm việc với bộ phận kiểm thử. Ban đầu em thấy hơi khó chịu khi phiếu bị trả lại, về sau em hiểu họ đang làm phần việc em khó tự làm được: em luôn thử theo đúng cách em hình dung khi viết mã, còn họ thử theo cách người dùng thật hành xử. Rất nhiều lỗi họ tìm ra là những trường hợp em không nghĩ tới.

Thứ tư là chấp nhận bỏ đi phần việc đã làm. Lần em huỷ bỏ toàn bộ phần thống kê đã hiện thực ở một dịch vụ để làm lại ở dịch vụ khác là lần khó chịu nhất, nhưng cũng là lần em trưởng thành nhất về nghề, vì công sức đã bỏ ra không phải lý do để giữ một quyết định sai.

Những bài học em cho là quan trọng nhất

1. Một tính năng chưa xong khi nó chạy được. Nó xong khi ta đã đi tìm và xử lý những chỗ nó âm thầm làm sai.
2. Phạm vi của một thay đổi quan trọng ngang với nội dung của nó. Một lỗi hẹp nên được sửa bằng một thay đổi hẹp, còn dọn mã phải là một việc riêng có kế hoạch

riêng.

3. Mọi tầng trừu tượng đều có giá. Viết truy vấn bằng ngôn ngữ lập trình rất tiện, nhưng ở chỗ nóng thì phải biết câu lệnh thật sự được gửi đi là gì.
4. Việc chặn phải nằm ở phía máy chủ. Ẩn một cái nút trên giao diện là trải nghiệm người dùng, không phải bảo mật.
5. Phần tính toán nên nằm ở nơi sở hữu dữ liệu, vì hai chỗ cùng định nghĩa một chỉ tiêu là một quả bom hẹn giờ. Và khi cùng một hiện tượng xuất hiện ở nhiều chỗ, rất có thể đó không phải nhiều lỗi mà là một vấn đề về thiết kế.
6. Hãy viết phần quay lại trước khi cần đến nó. Điều cần chuẩn bị không phải cho trường hợp mình sai, mà cho trường hợp mình không có thời gian tìm hiểu vì sao mình sai.
7. Kiểm thử có giá trị nhất là kiểm thử bảo vệ hành vi cũ, chứ không phải kiểm thử lại đúng cái mình vừa viết. Và hiệu năng là một thói quen viết mã, không phải một giai đoạn làm sau.

Những điều em chưa làm được

Một báo cáo chỉ kể phần tốt thì không trung thực, nên em xin ghi lại cả những hạn chế của mình. Em vẫn ước lượng thời gian kém, có phiếu nói hai ngày mà làm bốn ngày, và phần vượt gần như luôn nằm ở khâu đọc hiểu phần đã có chứ không nằm ở khâu viết mã. Em viết kiểm thử cũng chưa đồng đều: với những phần logic rõ ràng thì khá đầy đủ, nhưng với những phần chủ yếu là hiển thị thì em hay bỏ qua và chỉ thử bằng tay. Em còn phụ thuộc vào việc được hỏi, vì nhiều lần em giải quyết được vấn đề sau khi anh hướng dẫn đặt lại câu hỏi theo một hướng khác, còn em thì chưa tự đặt được những câu hỏi như vậy cho chính mình. Cuối cùng, em hiểu hệ thống theo từng mảng chứ chưa hiểu toàn cục: những phần mình đã chạm vào thì nắm khá rõ, nhưng nếu được hỏi về một phân hệ chưa làm thì vẫn phải mở mã ra đọc lại từ đầu.

b. Hướng phát triển tiếp theo

Tính năng khoá phòng hiện xử lý tốt trường hợp khoá theo một khoảng thời gian cụ thể. Hướng mở rộng tự nhiên là khoá theo chu kỳ lặp lại, ví dụ bảo trì định kỳ hằng tháng, và gắn lượt khoá với một kế hoạch bảo trì để người quản lý thấy được lịch sử sửa chữa của từng phòng; phần sinh lệnh theo sai khác trạng thái mà em đã viết có thể dùng lại cho hướng này mà không phải viết lại từ đầu.

Phần tính toán thống kê hiện chạy trực tiếp mỗi lần người dùng xem báo cáo, nên với những cơ sở có lượng khách lớn và khoảng thời gian xem rộng thì sẽ dần chậm. Hướng cải thiện là tính trước theo từng ngày rồi lưu lại, khi người dùng xem một khoảng

thì chỉ cần cộng các ngày trong khoảng đó; vì phần tính toán đã được tách thành một lớp thuần nên việc chuyển đổi không ảnh hưởng tới quy tắc nghiệp vụ. Với phần tối ưu, em mới chỉ xử lý những chỗ đã có người báo là chậm, nên việc nên làm tiếp là đo đạc chủ động để tìm ra những chỗ chậm mà chưa ai kịp phàn nàn. Với phần kiểm thử, khoảng trống rõ nhất là phần hiển thị.

Về bản thân, em muốn tiếp tục làm ở vị trí có thể chạm vào cả phần giao diện và phần máy chủ, vì sáu tháng qua cho em thấy nhiều vấn đề chỉ nhìn ra được khi ta thấy cả hai đầu. Ba việc cụ thể em muốn làm tiếp là học kỹ hơn về cơ sở dữ liệu, cụ thể là cách đọc kế hoạch thực thi của câu truy vấn và cách thiết kế chỉ mục; tập viết kiểm thử trước khi viết mã cho những phần có logic rõ; và tập tự trả lời câu hỏi thay đổi này sẽ ảnh hưởng tới những đâu trước khi viết dòng mã đầu tiên. Sáu tháng qua em thường trả lời được câu hỏi đó sau khi đã làm xong, và khoảng cách giữa sau khi làm xong và trước khi bắt đầu chính là khoảng cách em cần đi tiếp.

Sáu tháng ở KiotViet không dạy em thêm nhiều cú pháp hay công cụ. Nó dạy em cách làm việc với một hệ thống mình không viết ra, đang có người dùng thật, và không cho phép mình làm hỏng. Đó là thứ khó tự học ở nhà nhất.

Tài liệu tham khảo

[1] Robert C. Martin, *Clean Architecture*, Prentice Hall, 2017.

[2] Michael Feathers, *Working Effectively with Legacy Code*, Prentice Hall, 2004.

[3] T. Cormen, C. Leiserson, R. Rivest, C. Stein, *Introduction to Algorithms*, MIT Press, 2022.

[4] Martin Fowler, *CQRS* và *Command Query Separation*, martinfowler.com.

[5] Tài liệu Angular, mục *Change Detection* và *Pipes*, angular.dev.

[6] Tài liệu Microsoft về .NET và Entity Framework, learn.microsoft.com.

[7] Tài liệu Microsoft SQL Server, mục *Execution Plans* và *Locking*.

[8] Tài liệu Redis, mục *Distributed Locks* và *Key Expiration*.

[9] MDN Web Docs, mục *Unicode character class escapes*, developer.mozilla.org.

[10] Tài liệu xUnit.net và Jasmine về cách tổ chức trường hợp kiểm thử.

Ý kiến đánh giá:

. . . . . .

. . . . . .

. . . . . .

. . . . . .

. . . . . .

. . . . . .

. . . . . .

. . . . . .

Hà Nội, ngày tháng năm 20 .

Người hướng dẫn

(Ký, ghi rõ họ tên & dấu công ty)



Ý kiến đánh giá:

. . . . . .

. . . . . .

. . . . . .

. . . . . .

. . . . . .

. . . . . .

. . . . . .

. . . . . .

Điểm số: . . . . . Điểm chữ: . . . . . .

Hà Nội, ngày tháng năm 20 .

Giảng viên đánh giá

(Ký, ghi rõ họ tên)